llm_task1_enhanced_house_price_prediction20260308

March 14, 2026

0.1 Phase 1: Data Preparation & Understanding

[ ]:
```
# Import necessary libraries
import pandas as pd
import numpy as np
import re
import os
import math
import json
from datetime import datetime
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import ftfy

# Define file paths
# Get the current directory (where the notebook is located)
current_dir = os.path.dirname(os.path.abspath('__file__')) if '__file__' in␣
 ↪globals() else os.getcwd()

# Construct file paths
data_file = os.path.join(current_dir, 'data', 'realestate_data_london_2024_nov.
 ↪csv')
output_dir = os.path.join(current_dir, 'output')
output_file = os.path.join(output_dir, 'df_cleaned.csv')

# Create output directory if it doesn't exist
os.makedirs(output_dir, exist_ok=True)
```

0.1.1 1.1 Load and Explore the Dataset

[ ]:
```
# Load the dataset
print("Loading dataset...")
try:
    df = pd.read_csv(data_file, encoding="utf-8")
    print(f" Dataset loaded successfully from: {data_file}")
except FileNotFoundError:
    print(f" Error: File not found at {data_file}")
    print("Current working directory:", os.getcwd())
```

```
    print("Available files in data directory:")
    data_dir = os.path.join(current_dir, 'data')
    if os.path.exists(data_dir):
        print(os.listdir(data_dir))
    raise
```

[ ]:
```
# Display initial dataset information
print(f"\n1. Dataset shape: {df.shape}")
print(f"   Rows: {df.shape[0]}, Columns: {df.shape[1]}")

print(f"\n2. Columns:")
for i, col in enumerate(df.columns.tolist(), 1):
    print(f"   {i:2d}. {col}")

print(f"\n3. Missing values check:")
missing_values = df.isnull().sum()
if missing_values.sum() > 0:
    print("   Missing values found:")
    for col, missing_count in missing_values[missing_values > 0].items():
        missing_percent = (missing_count / len(df)) * 100
        print(f"   - {col}: {missing_count} missing ({missing_percent:.2f}%)")
else:
    print("    No missing values found in any column")

print(f"\n4. Data types:")
print(df.dtypes)

print(f"\n5. First 3 rows of original data:")
print(df.head(3))
```

0.1.2 1.2 Clean data

[ ]:
```
# Data Cleaning Functions
def clean_date(date_str):
    """
    Return '2024' for ALL rows, completely ignoring the original value
    """
    return '2024'  # Always return "2024" for all rows

def clean_description(html_text):
    """Clean description - remove HTML tags and extra whitespace"""
    # Handle missing values - return empty string instead of removing row
    if pd.isna(html_text):
        return ""

    # Convert to string
    text = str(html_text)
```

```
    # Remove HTML tags (preserve content between tags)
    text = re.sub(r'<[^>]+>', ' ', text)

    # Replace common HTML entities
    html_entities = {
        ' ': ' ',
        '&amp;': '&',
        '&lt;': '<',
        '&gt;': '>',
        '&quot;': '"',
        '&#39;': "'",
        '&rsquo;': "'",
        '&lsquo;': "'",
        '&rdquo;': '"',
        '&ldquo;': '"'
    }

    for entity, replacement in html_entities.items():
        text = text.replace(entity, replacement)

    # Clean up extra whitespace
    text = re.sub(r'\s+', ' ', text)

    # Strip leading/trailing whitespace
    return text.strip()

def clean_price(price_value):
    """Clean price - remove currency symbols and commas, convert to float"""
    # Handle missing values - return NaN but don't remove row
    if pd.isna(price_value):
        return np.nan

    # Convert to string
    price_str = str(price_value)

    # Extract all numbers (including decimals)
    # This preserves the numeric value regardless of format
    numbers = re.findall(r'[\d,\.]+', price_str)

    if not numbers:
        return np.nan

    # Take the first number found (should be the price)
    price_num = numbers[0]

    # Clean the number
```

```
    # Remove commas (thousands separators)
    price_num = price_num.replace(',', '')

    # Handle cases where dot might be decimal separator
    # If there's a dot and it's not the last character, assume it's decimal
    if '.' in price_num and price_num.rfind('.') < len(price_num) - 1:
        # Already has decimal point, keep as is
        pass
    else:
        # No valid decimal point found
        pass

    # Convert to float
    try:
        return float(price_num)
    except:
        # If conversion fails, try to handle special cases
        try:
            # Remove any remaining non-numeric characters
            clean_num = re.sub(r'[^\d\.]', '', price_str)
            return float(clean_num) if clean_num else np.nan
        except:
            return np.nan
```

[ ]:
```
# Apply data cleaning
# Create a copy for cleaning
df_cleaned = df.copy()

# 1 Clean and rename 'addedOn' column
print("\n1. Cleaning 'addedOn' column...")
df_cleaned['Date'] = df_cleaned['addedOn'].apply(clean_date)
df_cleaned = df_cleaned.drop('addedOn', axis=1)

# 2 Clean and rename 'descriptionHtml' column
print("\n2. Cleaning 'descriptionHtml' column...")
df_cleaned['listingDescription'] = df_cleaned['descriptionHtml'].
 ↪apply(clean_description)
df_cleaned = df_cleaned.drop('descriptionHtml', axis=1)

# Calculate some statistics about the cleaned descriptions
desc_lengths = df_cleaned['listingDescription'].apply(len)
print(f"    HTML tags removed")
print(f"   Average description length: {desc_lengths.mean():.0f} characters")
print(f"   Min length: {desc_lengths.min()} characters")
print(f"   Max length: {desc_lengths.max()} characters")

# 3 Clean 'price' column
```

```
print("\n3. Cleaning 'price' column...")
original_price_sample = df_cleaned['price'].head(3).tolist()
df_cleaned['price'] = df_cleaned['price'].apply(clean_price)

# Remove rows with invalid prices
original_rows = len(df_cleaned)
df_cleaned = df_cleaned.dropna(subset=['price'])
rows_removed = original_rows - len(df_cleaned)

print(f"   Currency symbols and commas removed")
print(f"   Rows with invalid prices removed: {rows_removed}")
print(f"   Price range: £{df_cleaned['price'].min():,.2f} to␣
 ↪£{df_cleaned['price'].max():,.2f}")
print(f"   Average price: £{df_cleaned['price'].mean():,.2f}")

# 4 Remove rows with missing values in key numeric features
key_cols = ['sizeSqFeetMax', 'bedrooms', 'bathrooms']
rows_before_dropna = len(df_cleaned)
df_cleaned = df_cleaned.dropna(subset=key_cols)
rows_removed_nulls = rows_before_dropna - len(df_cleaned)
print(f"\n4. Rows with missing values in {key_cols} removed:␣
 ↪{rows_removed_nulls}")

# 5 Check other columns
print("\n5. Checking other columns...")

# Check for missing values in other columns
missing_after_clean = df_cleaned.isnull().sum()
if missing_after_clean.sum() > 0:
    print("   Missing values after cleaning:")
    for col, missing_count in missing_after_clean[missing_after_clean > 0].
 ↪items():
        print(f"   - {col}: {missing_count} missing")
else:
    print("    No missing values in cleaned data")
```

[ ]:
```
# Display cleaned dataset information
print(f"\n1. Cleaned dataset shape: {df_cleaned.shape}")
print(f"   Rows: {df_cleaned.shape[0]}, Columns: {df_cleaned.shape[1]}")

print(f"\n2. Cleaned columns:")
for i, col in enumerate(df_cleaned.columns.tolist(), 1):
    print(f"   {i:2d}. {col} (dtype: {df_cleaned[col].dtype})")

print(f"\n3. Sample of cleaned data (first 2 rows):")
print(df_cleaned.head(2))
```

```
print(f"\n4. Data types summary:")
print(df_cleaned.dtypes)

print(f"\n5. Basic statistics for numeric columns:")
numeric_cols = df_cleaned.select_dtypes(include=[np.number]).columns
if len(numeric_cols) > 0:
    print(df_cleaned[numeric_cols].describe())
else:
    print("   No numeric columns found")
```

0.1.3 1.3 Save cleaned data

[ ]:
```
# Save cleaned data
df_cleaned.to_csv(output_file, index=False)
print(f" Cleaned data saved to: {output_file}")
```

0.2 Phase 2: Use NuExtract-1.5 to extract phrases

[ ]:
```
# Phase 2: Use NuExtract - 1.5 to extract key phrases
# Import necessary libraries
import pandas as pd
import numpy as np
import re
import os
import math
import json
from datetime import datetime
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import ftfy
import gc

# 2.0 Configurate

current_dir = os.path.dirname(os.path.abspath("__file__")) if "__file__" in␣
 ↪globals() else os.getcwd()
data_file = os.path.join(current_dir, "data", "df_cleaned.csv")
output_dir = os.path.join(current_dir, "output")
os.makedirs(output_dir, exist_ok=True)
trunc_log_path = os.path.join(output_dir, "truncated_rows_log.txt")

model_name = "numind/NuExtract-1.5"
batch_size = 1
max_length = 2048
max_new_tokens = 512
chunk_size = 5  # process 5 rows at a time
```

```
final_file = os.path.join(output_dir, "df_final_with_extracted_features.csv")
json_file = os.path.join(output_dir, "raw_outputs.jsonl")

# 2.1 Load data

df_cleaned = pd.read_csv(data_file)
print("Loaded:", data_file)
print(df_cleaned.head())

# 2.2 Load NuExtract-1.5 on GPU

print("Loading model......")
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,
    trust_remote_code=True
).to(device).eval()

tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    trust_remote_code=True
)

print("Model and tokenizer loaded.")

# 2.3 JSON template and keyword lists

template_dict = {
    "luxury_features": "",
    "transport_mentions": "",
    "school_mentions": "",
    "renovation_mentions": ""
}
template_str = json.dumps(template_dict, indent=4)

luxury_keywords = [
    # ===== Level 1 (Ultra-luxury / top-tier signals) =====
    "penthouse", "most exclusive", "exclusive development", "epitome of␣
 ↪luxury", "unparalleled luxury",
    "luxury living", "world-class", "iconic", "coveted address", "prestigious␣
 ↪address", "prime position",
    "prime residential address", "mayfair", "knightsbridge", "belgravia",␣
 ↪"chelsea barracks",
    "panoramic views", "breathtaking views", "360º view",
```

```
    # ===== Level 2 (Luxury services / security / access control) =====
    "24-hour concierge", "concierge", "security", "first class security",␣
 ↪"gated", "secure gates",
    "secure", "private gated road", "private road",

    # ===== Level 3 (Luxury amenities / lifestyle facilities) =====
    "swimming pool", "spa", "gym", "gymnasium", "sauna", "steam room",␣
 ↪"treatment room", "cinema room",
    "home cinema", "wine cellar", "wine room", "billiards room", "personal␣
 ↪training facilities",
    "leisure facilities", "business centre", "private meeting rooms", "chef's␣
 ↪kitchen",

    # ===== Level 4 (High-end outdoor / layout / building features) =====
    "roof terrace", "private terrace", "terrace", "balcony", "private garden",␣
 ↪"landscaped garden",
    "floor-to-ceiling windows", "high ceilings", "lift", "passenger lift",␣
 ↪"glass lift", "garage",
    "secure parking",

    # ===== Level 5 (Interior quality / design / finishes) =====
    "bespoke", "bespoke joinery", "interior designed", "celebrated interior",␣
 ↪"high specification",
    "state-of-the-art", "state of the art", "very high standard", "finished to␣
 ↪the highest specification",
    "new benchmark of quality", "natural stone", "marble", "integrated␣
 ↪appliances", "underfloor heating",
    "air conditioning",

    # ===== Level 6 (Premium rooms / layout language) =====
    "master bedroom suite", "dressing room", "en-suite bathroom", "reception␣
 ↪rooms", "formal dining",
    "drawing room", "library", "study", "staff accommodation", "staff flat",␣
 ↪"self-contained staff lodge",
    "mews house", "porticoed entrance", "impressive entrance hall",

    # ===== Level 7 (Luxury adjectives / marketing words) =====
    "luxury", "luxurious",
]

transport_keywords = [
    # ===== Level 1: Stations / lines / rail / roads / airports =====
    "station", "underground", "tube", "overground", "rail", "train", "london␣
 ↪underground",
```

```
    "underground stations", "tube stations", "victoria station", "sloane square␣
 ↪station",
    "knightsbridge station", "hyde park corner station", "green park station",
    "notting hill gate station", "holland park station", "regent's park␣
 ↪station",
    "piccadilly line", "central line", "bakerloo line", "circle line",␣
 ↪"district line",
    "national rail services", "heathrow airport", "m4", "m3", "bus", "bus␣
 ↪routes",

    # ===== Level 2: Walking distance / access / connectivity phrases =====
    "transport links", "excellent transport links", "good transport links",␣
 ↪"fantastic travel links",
    "travel links", "road links", "motorway", "well-connected", "well␣
 ↪connected", "accessible",
    "easy access", "easy access to", "quick access", "excellent access",␣
 ↪"excellent connections",
    "good connections", "within walking distance", "walking distance", "short␣
 ↪walk", "just a short walk",
    "easy walking distance", "within easy reach of", "close to", "close to␣
 ↪transport",
    "well-positioned for", "moments from", "stone's throw away", "just a␣
 ↪stone's throw away",
    "minutes away", "minutes from", "less than a mile away", "approximately 0.3␣
 ↪miles away",
    "approximately 0.4 miles away"
]

school_keywords = [
    "good schools", "schools", "school", "excellent schools", "well-served by␣
 ↪excellent schools"
]

renovation_keywords = [
    "renovated", "newly renovated", "recently renovated",
    "refurbished", "newly refurbished", "recently refurbished",
    "modernised", "modernized", "upgraded", "redecorated",
    "refitted", "updated", "brand new", "rebuilt", "reconstructed",
    "remodeled", "redesigned", "reimagined", "renewed", "reconditioned",
    "excellent condition", "good condition", "immaculate condition",
    "turnkey", "move-in ready", "ready to move into",
    "restored", "redeveloped", "reconfigured",
    "completely refurbished", "fully refurbished",
    "finished", "new kitchen", "new bathrooms"
]
```

```
# Use a reduced set of strong luxury keywords in the prompt (no extra GPU/RAM␣
 ↪cost)
top_luxury_keywords = luxury_keywords[:40]

def build_prompt(text: str) -> str:
    instructions = f"""
You are extracting structured information from a London real-estate listing.

GENERAL RULES
- Read the ENTIRE listing carefully.
- For each field, return as MANY relevant key phrases as you can find (up to␣
 ↪8), separated by semicolons (;).
- Each phrase must be copied VERBATIM from the text (no paraphrasing).
- Do NOT invent information. If nothing relevant is found for a field, set that␣
 ↪field to "" (empty string).
- Avoid full sentences; use compact phrases only.
- If at least 3 relevant phrases exist for a field, return at least 3.

FIELD DEFINITIONS

1) "luxury_features":
   - Phrases that indicate luxury amenities, services, finishes, or exclusive␣
 ↪character.
   - Include EVERY phrase that includes or is clearly similar to any of these␣
 ↪keywords:
     {", ".join(top_luxury_keywords)}
   - Do not skip phrases that match these keywords, even if they seem␣
 ↪repetitive.
   - Examples of good outputs:
     "indoor swimming pool"; "spa with sauna and steam room"; "24 hour␣
 ↪concierge";
     "landscaped private garden"; "home cinema"; "temperature-controlled wine␣
 ↪cellar".

2) "transport_mentions":
   - Phrases that describe public transport, road access, or how easy it is to␣
 ↪reach key areas.
   - Include EVERY phrase that includes or is clearly similar to any of these␣
 ↪keywords:
     {", ".join(transport_keywords)}
   - Examples:
     "short walk to Knightsbridge Underground Station";
     "excellent transport links to the City and the West End";
     "within walking distance of Victoria Station".

3) "school_mentions":
```

```
   - Phrases that mention schools, school quality, or proximity to education.
   - Include EVERY phrase that includes or is clearly similar to any of these␣
 ↪keywords:
     {", ".join(school_keywords)}
   - Examples:
     "excellent local schools";
     "close to top independent schools";
     "within the catchment area of outstanding primary schools".

4) "renovation_mentions":
   - Phrases that describe renovation, refurbishment, modernisation, or␣
 ↪condition of the property.
   - Include EVERY phrase that includes or is clearly similar to any of these␣
 ↪keywords:
     {", ".join(renovation_keywords)}
   - Examples:
     "newly refurbished throughout";
     "recently renovated to a high specification";
     "turnkey condition";
     "comprehensively redeveloped".

OUTPUT FORMAT
- Return a single JSON object exactly matching this template:
{template_str}

- Each value must be a single string containing zero or more phrases separated␣
 ↪by semicolons.
- Do NOT add extra keys or commentary.
"""
    return f"""<|input|>
{instructions.strip()}

### Text:
{text}

<|output|>"""

# 2.4 Append the truncated row into log file

def log_truncation(idx, original_len, kept_len):
    with open(trunc_log_path, "a", encoding="utf-8") as f:
        f.write(
            f"[{datetime.now().isoformat()}] "
            f"row_index={idx}, original_chars={original_len},␣
 ↪kept_chars={kept_len}\n"
        )
```

```python
# 2.5 Run NuExtract on a batch of texts

def nuextract_batch(texts, row_indices, batch_size, max_length, max_new_tokens):
    device_local = model.device
    prompts = []
    # track which prompts were truncated at character level
    for idx, t in zip(row_indices, texts):
        prompt = build_prompt(t)
        # rough char-length check before tokenization
        if len(prompt) > max_length * 4:  # heuristic: ~4 chars per token
            log_truncation(idx, len(prompt), max_length * 4)
            # keep the FIRST part (instructions + start of listing) to reduce␣
 ↪confusion
            prompt = prompt[:max_length * 4]
        prompts.append(prompt)

    outputs = []

    with torch.no_grad():
        for i in range(0, len(prompts), batch_size):
            batch_prompts = prompts[i:i + batch_size]

            enc = tokenizer(
                batch_prompts,
                return_tensors="pt",
                truncation=True,
                padding=True,
                max_length=max_length
            ).to(device_local)

            pred_ids = model.generate(
                **enc,
                max_new_tokens=max_new_tokens,
                do_sample=False,
                use_cache=True
            )

            decoded = tokenizer.batch_decode(pred_ids, skip_special_tokens=True)

            for out in decoded:
                if "<|output|>" in out:
                    outputs.append(out.split("<|output|>", 1)[1].strip())
                else:
                    outputs.append(out.strip())
    return outputs

# 2.6 Robust JSON extraction (one line per input) + hybrid merge
```

```python
empty_obj = {
    "luxury_features": "",
    "transport_mentions": "",
    "school_mentions": "",
    "renovation_mentions": ""
}

def extract_last_json(text: str):
    # Take the model output and: 1) find the last {...} block 2) parse it as␣
 ↪JSON 3) ensure all expected keys exist.
    start = text.rfind("{")
    end = text.rfind("}")
    if start == -1 or end == -1 or end <= start:
        return empty_obj.copy(), None

    candidate = text[start:end+1]
    # collapse whitespace to keep it one line
    candidate_one_line = re.sub(r"\s+", " ", candidate).strip()

    try:
        obj = json.loads(candidate_one_line)
    except json.JSONDecodeError:
        return empty_obj.copy(), None

    # Ensure keys
    for k in empty_obj.keys():
        obj.setdefault(k, "")

    return obj, candidate_one_line

# Simple keyword matcher (runs on CPU; very cheap)
def find_keyword_phrases(text: str, keywords):
    if not isinstance(text, str):
        return []
    t = text.lower()
    hits = []
    for kw in keywords:
        if kw.lower() in t:
            hits.append(kw)
    # remove duplicates while preserving order
    seen = set()
    unique_hits = []
    for h in hits:
        if h not in seen:
            seen.add(h)
            unique_hits.append(h)
```

```
    return unique_hits

def merge_llm_and_keywords(listing_text: str, obj: dict, max_phrases: int = 8)␣
 ↪-> dict:
    # Helper to split current semicolon string into list
    def split_field(value: str):
        if not isinstance(value, str):
            return []
        return [x.strip() for x in value.split(";") if x.strip()]

    # 1) luxury
    lux_from_llm = split_field(obj.get("luxury_features", ""))
    lux_kw_hits = find_keyword_phrases(listing_text, luxury_keywords)
    lux_merged = []
    for x in lux_from_llm + lux_kw_hits:
        if x not in lux_merged:
            lux_merged.append(x)
    obj["luxury_features"] = "; ".join(lux_merged[:max_phrases])

    # 2) transport
    trans_from_llm = split_field(obj.get("transport_mentions", ""))
    trans_kw_hits = find_keyword_phrases(listing_text, transport_keywords)
    trans_merged = []
    for x in trans_from_llm + trans_kw_hits:
        if x not in trans_merged:
            trans_merged.append(x)
    obj["transport_mentions"] = "; ".join(trans_merged[:max_phrases])

    # 3) schools
    school_from_llm = split_field(obj.get("school_mentions", ""))
    school_kw_hits = find_keyword_phrases(listing_text, school_keywords)
    school_merged = []
    for x in school_from_llm + school_kw_hits:
        if x not in school_merged:
            school_merged.append(x)
    obj["school_mentions"] = "; ".join(school_merged[:max_phrases])

    # 4) renovation
    reno_from_llm = split_field(obj.get("renovation_mentions", ""))
    reno_kw_hits = find_keyword_phrases(listing_text, renovation_keywords)
    reno_merged = []
    for x in reno_from_llm + reno_kw_hits:
        if x not in reno_merged:
            reno_merged.append(x)
    obj["renovation_mentions"] = "; ".join(reno_merged[:max_phrases])

    return obj
```

```
# 2.7 Loop over dataset in chunks of chunk_size rows to run model

# 1) Write CSV header once (empty file with header)
if not os.path.exists(final_file):
    sample_df = df_cleaned.iloc[:1, :]
    dummy_extracted = pd.DataFrame([empty_obj])
    dummy_final = pd.concat([sample_df.reset_index(drop=True),␣
 ↪dummy_extracted], axis=1)
    dummy_final.iloc[0:0].to_csv(final_file, index=False)  # write only header

# 2) Ensure JSONL file exists but DO NOT clear if you want resume
if not os.path.exists(json_file):
    open(json_file, "w", encoding="utf-8").close()

n_rows = len(df_cleaned)

# Count how many rows were already processed (1 line per row)
with open(json_file, "r", encoding="utf-8") as f:
    processed_count = sum(1 for _ in f)

print("Already processed rows:", processed_count)

resume_from = processed_count  # first row index to process next

for start in range(resume_from, n_rows, chunk_size):  # use n_rows to replace␣
 ↪500 after testing
    end = min(start + chunk_size, n_rows)

    if torch.cuda.is_available():
        free_mem, total_mem = torch.cuda.mem_get_info()
        print(f"GPU Memory Free: {free_mem / 1024**2:.2f} MB")
    print(f"Processing rows {start} to {end - 1}")
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    df_sample = df_cleaned.iloc[start:end, :]
    texts = df_sample["listingDescription"].fillna("").astype(str).tolist()
    row_indices = df_sample.index.tolist()

    print("Running NuExtract on", len(texts), "descriptions...")
    raw_outputs = nuextract_batch(
        texts,
        row_indices,
        batch_size=batch_size,
        max_length=max_length,
```

```
        max_new_tokens=max_new_tokens,
    )
    print("Got outputs:", len(raw_outputs))

    for i, out in enumerate(raw_outputs):
        print(f"RAW {i}:", out)

    # Parse JSON, one line per input, then apply hybrid merge
    parsed = []
    clean_json_strings = []

    for raw_text, listing_text in zip(raw_outputs, texts):
        obj, clean_str = extract_last_json(raw_text)
        # hybrid merge with deterministic keyword hits
        obj = merge_llm_and_keywords(listing_text, obj, max_phrases=8)

        parsed.append(obj)
        if clean_str is None:
            clean_str = json.dumps(obj, ensure_ascii=False)
        clean_json_strings.append(clean_str)

    # Append JSONL
    with open(json_file, "a", encoding="utf-8") as f:
        for line in clean_json_strings:
            f.write(line.strip() + "\n")

    print("Appended to JSONL:", json_file)

    # Build and append df_final chunk
    df_extracted = pd.DataFrame(parsed)
    df_final_chunk = pd.concat([df_sample.reset_index(drop=True),␣
↪df_extracted], axis=1)
    print("df_final_chunk:\n", df_final_chunk)

    df_final_chunk.to_csv(
        final_file,
        mode="a",
        index=False,
        header=False,  # header already written once
    )

    # Free Python & CUDA memory before next chunk
    del raw_outputs, df_extracted, df_final_chunk
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
```

0.3 Phase 3: Convert Extracted Phrases to Feature Scores

[ ]:
```
# Phase 3: Convert Extracted Phrases to Feature Scores
import os
import pandas as pd
import numpy as np

# 3.0 Reuse the same keyword lists as in the NuExtract script
luxury_keywords = [

    # ===== Level 1 (Ultra-luxury / top-tier signals) =====
    "penthouse", "most exclusive", "exclusive development", "epitome of␣
↪luxury", "unparalleled luxury",
    "luxury living", "world-class", "iconic", "coveted address", "prestigious␣
↪address", "prime position",
    "prime residential address", "mayfair", "knightsbridge", "belgravia",␣
↪"chelsea barracks",
    "panoramic views", "breathtaking views", "360º view",

    # ===== Level 2 (Luxury services / security / access control) =====
    "24-hour concierge", "concierge", "security", "first class security",␣
↪"gated", "secure gates",
    "secure", "private gated road", "private road",

    # ===== Level 3 (Luxury amenities / lifestyle facilities) =====
    "swimming pool", "spa", "gym", "gymnasium", "sauna", "steam room",␣
↪"treatment room", "cinema room",
    "home cinema", "wine cellar", "wine room", "billiards room", "personal␣
↪training facilities",
    "leisure facilities", "business centre", "private meeting rooms", "chef's␣
↪kitchen",

    # ===== Level 4 (High-end outdoor / layout / building features) =====
    "roof terrace", "private terrace", "terrace", "balcony", "private garden",␣
↪"landscaped garden",
    "floor-to-ceiling windows", "high ceilings", "lift", "passenger lift",␣
↪"glass lift", "garage",
    "secure parking",

    # ===== Level 5 (Interior quality / design / finishes) =====
    "bespoke", "bespoke joinery", "interior designed", "celebrated interior",␣
↪"high specification",
    "state-of-the-art", "state of the art", "very high standard", "finished to␣
↪the highest specification",
    "new benchmark of quality", "natural stone", "marble", "integrated␣
↪appliances", "underfloor heating",
    "air conditioning",
```

```
    # ===== Level 6 (Premium rooms / layout language) =====
    "master bedroom suite", "dressing room", "en-suite bathroom", "reception␣
 ↪rooms", "formal dining",
    "drawing room", "library", "study", "staff accommodation", "staff flat",␣
 ↪"self-contained staff lodge",
    "mews house", "porticoed entrance", "impressive entrance hall",

    # ===== Level 7 (Luxury adjectives / marketing words) =====
    "luxury", "luxurious",
]

transport_keywords = [

    # ===== Level 1: Stations / lines / rail / roads / airports =====
    "station", "underground", "tube", "overground", "rail", "train", "london␣
 ↪underground",
    "underground stations", "tube stations", "victoria station", "sloane square␣
 ↪station",
    "knightsbridge station", "hyde park corner station", "green park station",
    "notting hill gate station", "holland park station", "regent's park␣
 ↪station",
    "piccadilly line", "central line", "bakerloo line", "circle line",␣
 ↪"district line",
    "national rail services", "heathrow airport", "m4", "m3", "bus", "bus␣
 ↪routes",

    # ===== Level 2: Walking distance / access / connectivity phrases =====
    "transport links", "excellent transport links", "good transport links",␣
 ↪"fantastic travel links",
    "travel links", "road links", "motorway", "well-connected", "well␣
 ↪connected", "accessible",
    "easy access", "easy access to", "quick access", "excellent access",␣
 ↪"excellent connections",
    "good connections", "within walking distance", "walking distance", "short␣
 ↪walk", "just a short walk",
    "easy walking distance", "within easy reach of", "close to", "close to␣
 ↪transport",
    "well-positioned for", "moments from", "stone's throw away", "just a␣
 ↪stone's throw away",
    "minutes away", "minutes from", "less than a mile away", "approximately 0.3␣
 ↪miles away",
    "approximately 0.4 miles away"
]

school_keywords = [
```

```
    "good schools", "schools", "school", "excellent schools", "well-served by␣
 ↪excellent schools"
]

renovation_keywords = [
    "renovated", "newly renovated", "recently renovated",
    "refurbished", "newly refurbished", "recently refurbished",
    "modernised", "modernized", "upgraded", "redecorated",
    "refitted", "updated", "brand new", "rebuilt", "reconstructed",
    "remodeled", "redesigned", "reimagined", "renewed", "reconditioned",
    "excellent condition", "good condition", "immaculate condition",
    "turnkey", "move-in ready", "ready to move into",
    "restored", "redeveloped", "reconfigured",
    "completely refurbished", "fully refurbished",
    "finished", "new kitchen", "new bathrooms"
]

# Lowercased versions for matching
luxury_kw_lc     = [k.lower() for k in luxury_keywords]
transport_kw_lc  = [k.lower() for k in transport_keywords]
school_kw_lc     = [k.lower() for k in school_keywords]
renovation_kw_lc = [k.lower() for k in renovation_keywords]

# 3.1 Load df_final_with_extracted_features.csv
current_dir = os.path.dirname(os.path.abspath("__file__")) if "__file__" in␣
 ↪globals() else os.getcwd()
output_dir = os.path.join(current_dir, "output")
input_file = os.path.join(output_dir, "df_final_with_extracted_features.csv")
df = pd.read_csv(input_file)
print("Loaded:", input_file)
print(df[["luxury_features", "transport_mentions", "school_mentions",␣
 ↪"renovation_mentions"]].head())

# 3.2 Define rules to transfer phrases into scores

# Function: check if cell has any non-empty text
def has_text(x):
    if isinstance(x, str):
        return x.strip() != ""
    return False

# helper: count keyword occurrences in text
def count_matches(text, keyword_list):
    if not isinstance(text, str) or text.strip() == "":
        return 0
    t = text.lower()
    return sum(1 for k in keyword_list if k in t)
```

```python
# 3.2.1 luxury_features: use renovation_keywords with different weights for first 20 vs rest
def score_luxury(text):
    if not has_text(text):
        return 0

    t = text.lower()

    # split renovation keywords into first 20 and remaining
    first_part = renovation_kw_lc[:20]
    later_part = renovation_kw_lc[20:]

    count_first = sum(1 for k in first_part if k in t)
    count_later = sum(1 for k in later_part if k in t)

    # +1 per 1 key phrase in first 20
    score = count_first *1

    # +1 per 5 key phrases after first 20
    score += count_later // 5

    return score

# 3.2.2 transport_mentions: +1 per 5 key phrases in transport_keywords
def score_transport(text):
    if not has_text(text):
        return 0

    match_count = count_matches(text, transport_kw_lc)
    score = match_count // 5
    return score

# 3.2.3 school_mentions: +1 per 2 key phrases in school_keywords
def score_school(text):
    if not has_text(text):
        return 0

    match_count = count_matches(text, school_kw_lc)
    score = match_count // 2
    return score

# 3.2.4 renovation_mentions: +1 per 5 key phrases in renovation_keywords
def score_renovation(text):
    if not has_text(text):
        return 0
```

```
    match_count = count_matches(text, renovation_kw_lc)
    score = match_count // 5
    return score

# 3.3 Apply scoring and drop original phrase columns
df["luxury_score"] = df["luxury_features"].apply(score_luxury)
df["transport_score"] = df["transport_mentions"].apply(score_transport)
df["school_score"] = df["school_mentions"].apply(score_school)
df["renovation_score"] = df["renovation_mentions"].apply(score_renovation)

df = df.drop(columns=["luxury_features", "transport_mentions",␣
 ↪"school_mentions", "renovation_mentions"])

# 3.4 Save as a new DataFrame/file
output_file = os.path.join(output_dir, "df_with_extracted_scores.csv")
df.to_csv(output_file, index=False)
print("Saved with scores to:", output_file)

print(df[["price", "luxury_score", "transport_score", "school_score",␣
 ↪"renovation_score"]].head())
```

0.4 Phase 4: Train & Evaluate Random Forest Models

- Baseline: df_cleaned.csv (no extracted scores)
- Enhanced: df_with_extracted_scores.csv (with extracted scores)

[1]:
```
# Phase 4: Train & Evaluate Random Forest Models

import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
import joblib

# Function: evaluate model
def evaluate_model(model, X_train, X_test, y_train, y_test, label="model"):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    print(f"\n=== {label} ===")
    print(f"RMSE: {rmse:,.2f}")
    print(f"MAE : {mae:,.2f}")
    return rmse, mae
```

```
# 4.1 Baseline model on df_cleaned.csv
current_dir = os.path.dirname(os.path.abspath("__file__")) if "__file__" in␣
 ↪globals() else os.getcwd()
output_dir = os.path.join(current_dir, "output")
cleaned_file = os.path.join(output_dir, "df_cleaned.csv")
df_cleaned = pd.read_csv(cleaned_file)
print("Loaded cleaned data:", cleaned_file)
print("Cleaned columns:", df_cleaned.columns.tolist())

target_col = "price"

# numeric features
num_cols = ["sizeSqFeetMax", "bedrooms", "bathrooms"]
# categorical to encode
cat_cols = ["propertyType", "listingUpdateReason"]

# one-hot encode categoricals
df_cleaned_cat = pd.get_dummies(df_cleaned[cat_cols], prefix=cat_cols,␣
 ↪drop_first=False)
X_base = pd.concat([df_cleaned[num_cols].reset_index(drop=True),
                    df_cleaned_cat.reset_index(drop=True)], axis=1)
y_base = df_cleaned[target_col].copy()

Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_base, y_base, test_size=0.2, random_state=42
)

rf_base = RandomForestRegressor(
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)

rmse_base, mae_base = evaluate_model(
    rf_base, Xb_train, Xb_test, yb_train, yb_test,
    label="Random Forest (baseline: num + encoded propertyType/
 ↪listingUpdateReason)"
)

baseline_model_path = os.path.join(output_dir, "rf_baseline_model.pkl")
joblib.dump(rf_base, baseline_model_path)
print("Baseline model saved to:", baseline_model_path)

# 4.2 Enhanced model on df_with_extracted_scores.csv
#    (must already contain luxury_score, transport_score, school_score,␣
 ↪renovation_score)
scores_file = os.path.join(output_dir, "df_with_extracted_scores.csv")
```

```
df_scores = pd.read_csv(scores_file)
print("\nLoaded data with extracted scores:", scores_file)
print("Columns:", df_scores.columns.tolist())

# numeric + scores
score_cols = ["luxury_score", "transport_score", "school_score",␣
 ↪"renovation_score"]
num_cols_scores = ["sizeSqFeetMax", "bedrooms", "bathrooms"] + score_cols

# one-hot encode same categoricals
df_scores_cat = pd.get_dummies(df_scores[cat_cols], prefix=cat_cols,␣
 ↪drop_first=False)

X_scores = pd.concat([df_scores[num_cols_scores].reset_index(drop=True),
                      df_scores_cat.reset_index(drop=True)], axis=1)
y_scores = df_scores[target_col].copy()

Xs_train, Xs_test, ys_train, ys_test = train_test_split(
    X_scores, y_scores, test_size=0.2, random_state=42
)

rf_scores = RandomForestRegressor(
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)

rmse_scores, mae_scores = evaluate_model(
    rf_scores, Xs_train, Xs_test, ys_train, ys_test,
    label="Random Forest (num + encoded categoricals + extracted scores)"
)

scores_model_path = os.path.join(output_dir, "rf_with_scores_model.pkl")
joblib.dump(rf_scores, scores_model_path)
print("Model with scores saved to:", scores_model_path)

# 4.3 Compare performance
print("\n=== Performance comparison (test set) ===")
print(f"Baseline RF - RMSE: {rmse_base:,.2f}, MAE: {mae_base:,.2f}")
print(f"With scores RF - RMSE: {rmse_scores:,.2f}, MAE: {mae_scores:,.2f}")

if rmse_scores < rmse_base:
    print("→ RMSE improved after adding extracted feature scores.")
else:
    print("→ RMSE did not improve after adding extracted feature scores.")

if mae_scores < mae_base:
```

```
    print("→ MAE improved after adding extracted feature scores.")
else:
    print("→ MAE did not improve after adding extracted feature scores.")
```

```
Loaded cleaned data: c:\Users\Admin\Python\S8_Thesis\llm\output\df_cleaned.csv
Cleaned columns: ['title', 'propertyType', 'sizeSqFeetMax', 'bedrooms',
'bathrooms', 'listingUpdateReason', 'price', 'Date', 'listingDescription']

=== Random Forest (baseline: num + encoded propertyType/listingUpdateReason) ===
RMSE: 5,428,669.93
MAE : 3,214,482.12
Baseline model saved to:
c:\Users\Admin\Python\S8_Thesis\llm\output\rf_baseline_model.pkl

Loaded data with extracted scores:
c:\Users\Admin\Python\S8_Thesis\llm\output\df_with_extracted_scores.csv
Columns: ['title', 'propertyType', 'sizeSqFeetMax', 'bedrooms', 'bathrooms',
'listingUpdateReason', 'price', 'Date', 'listingDescription', 'luxury_score',
'transport_score', 'school_score', 'renovation_score']

=== Random Forest (num + encoded categoricals + extracted scores) ===
RMSE: 3,077,608.38
MAE : 2,109,198.41
Model with scores saved to:
c:\Users\Admin\Python\S8_Thesis\llm\output\rf_with_scores_model.pkl

=== Performance comparison (test set) ===
Baseline RF - RMSE: 5,428,669.93, MAE: 3,214,482.12
With scores RF - RMSE: 3,077,608.38, MAE: 2,109,198.41
→ RMSE improved after adding extracted feature scores.
→ MAE improved after adding extracted feature scores.
```